BCS501 · SOFTWARE ENGINEERING & PROJECT MANAGEMENT

Module 3

Software Design Concepts

Design in the context of software engineering, the design process, core design concepts, and the design model.

Prepared By,
Dr.Manjunath H,
Assistant Professor, CSE,BIT

Textbook reference: TB1 Chapter 8 (8.1 – 8.4)

ROADMAP

What We Will Cover

8.1 **Design Within the Context of Software Engineering**

How design sits between requirements modeling and construction.

8.2 **The Design Process**

Software quality guidelines and attributes; the evolution of software design.

8.3 **Design Concepts**

Abstraction, architecture, patterns, modularity, information hiding, refinement, and more.

8.4 **The Design Model**

Data, architectural, interface, component-level, and deployment-level design elements.

From Requirements Model to Design Model

Requirements model

- Scenario-based elements
- Class-based elements
- Flow-oriented elements
- Behavioral elements

Design model

- Data / class design
- Architectural design
- Interface design
- Component-level design

Construction

- Code generation
- Testing
- Deployment feedback

- Software design is the core technical kernel of software engineering — it is applied regardless of the process model used.
- Design begins once requirements are modeled and builds the blueprint for construction and testing; it must stay traceable to requirements.

Translating Requirements into Design

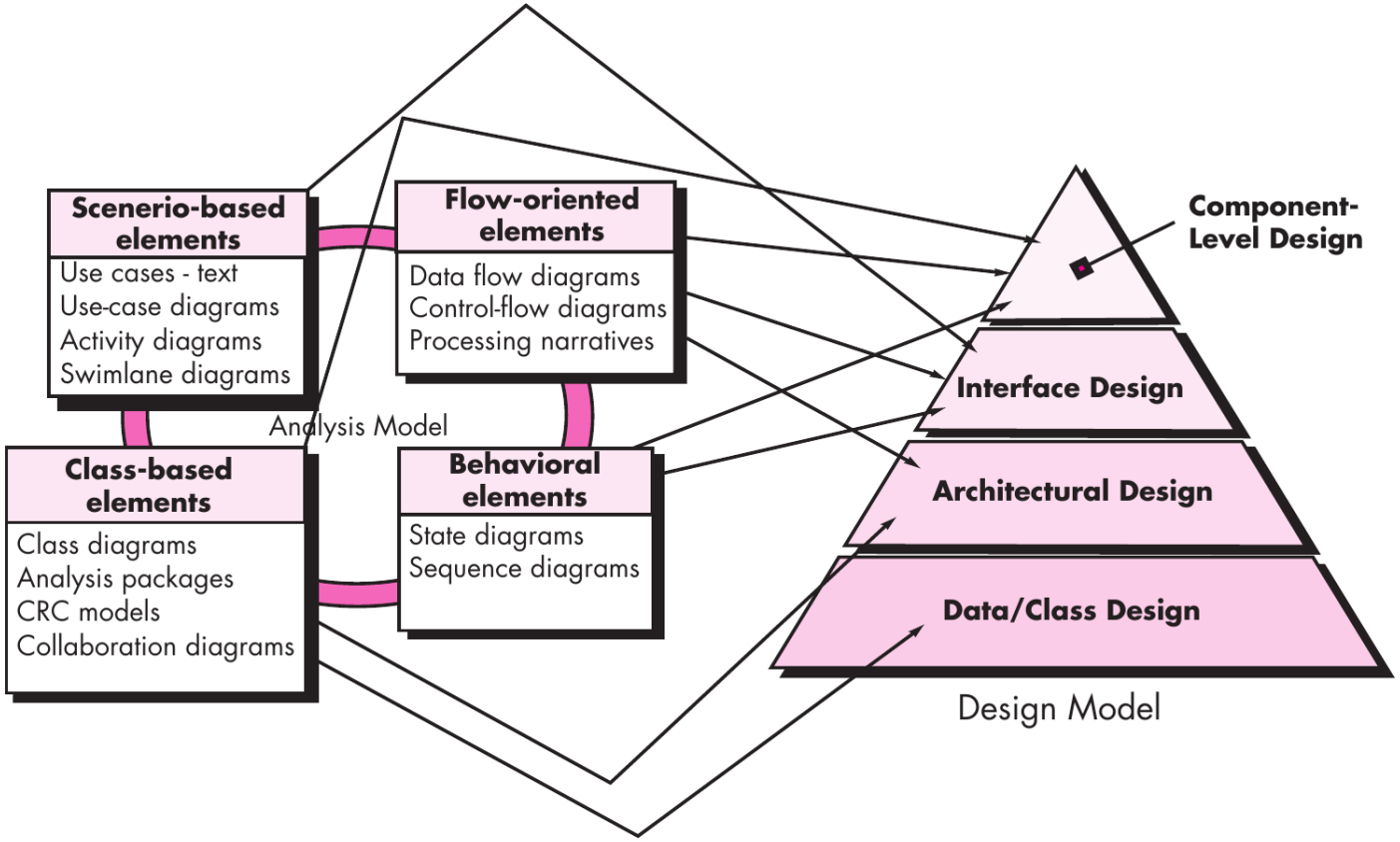


Figure 8.1 — Translating the requirements model into the design model (TB1, Ch. 8)

Reading the figure

- Four requirements-model elements (scenario-based, class-based, flow-oriented, behavioral) feed the design task.
- Design produces a data/class design, an architectural design, an interface design, and a component-level design.
- Class models are transformed into design-class realizations and data structures.
- The architectural design is derived from the requirements model and defines relationships among major structural elements.

Why Design Matters

1

Quality is established here

Design is where software quality is built in; it provides representations that can be assessed for quality before code exists.

2

Requirements become a product

Design is the only way to accurately translate stakeholder requirements into a finished software product or system.

3

Without design: unstable systems

Skipping design risks a system that breaks with small changes, is hard to test, and whose quality can’t be judged until late in the project.

An Iterative Refinement of Requirements



What the process does

- Transforms the requirements model into a blueprint for constructing software.
- Starts with a broad, architecture-level view and gradually refines it to lower abstraction.
- Is iterative — each refinement exposes errors and omissions to correct.

Three goals of a good design

- Implements all explicit requirements and the implicit needs of stakeholders.
- Is a readable, understandable guide for those who code, test, and later maintain it.
- Gives a complete picture of the software — data, functional, and behavioral domains — from an implementation view.

Where Design Fits in the Life Cycle

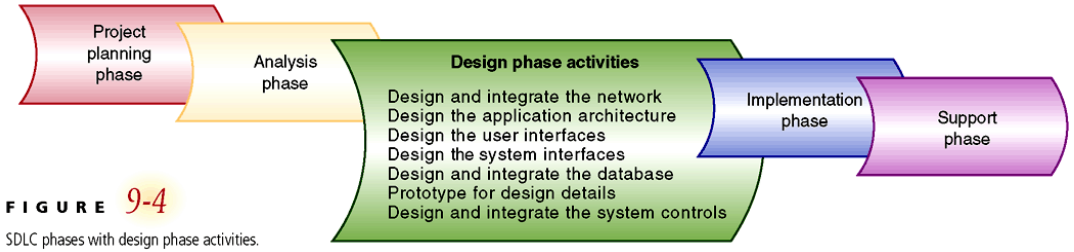


FIGURE 9-4
SDLC phases with design phase activities.

Design phase within the SDLC phases (Ch09)

- Design follows analysis and precedes implementation in the life cycle.
- Design phase activities: network, application architecture, user and system interfaces, database, prototypes for design details, and system controls.

FIGURE 9-5
Design phase activities and key questions.

Design phase activity	Key question
Design and integrate the network	*Have we specified in detail how the various parts of the system will communicate with each other throughout the organization?*
Design the application architecture	*Have we specified in detail how each system activity is actually carried out by the people and computers?*
Design the user interface(s)	*Have we specified in detail how all users will interact with the system?*
Design the system interface(s)	*Have we specified in detail how the system will work with all other systems inside and outside our organization?*
Design and integrate the database	*Have we specified in detail how and where the system will store all of the information needed by the organization?*
Prototype for design details	*Have we created prototypes to ensure all detailed design decisions have been fully understood?*
Design and integrate the system controls	*Have we specified in detail how we can be sure that the system operates correctly and the data maintained by the system are safe and secure?*

Design phase activities and the key question for each (Ch09)

Guidelines for a Good Design

Sound architecture

Uses recognizable architectural styles or patterns, built from components with good design characteristics, and can be implemented evolutionarily.

Modular

Logically partitioned into elements or subsystems.

Distinct representations

Separate representations of data, architecture, interfaces, and components.

Appropriate data structures

Drawn from recognizable patterns and suited to the classes being implemented.

Independent components

Components have independent functional characteristics.

Simple interfaces

Interfaces reduce the complexity of connections between components and the outside world.

Repeatable method

Derived using a method driven by information from requirements analysis.

Meaningful notation

The representation effectively communicates the design's meaning.

Quality Attributes — FURPS

F

Functionality

Assessed by the feature set and capabilities of the program, the generality of functions, and security of the overall system.

U

Usability

Assessed by human factors, overall aesthetics, consistency, and documentation.

R

Reliability

Measured by frequency and severity of failure, accuracy of output, mean-time-to-failure, ability to recover, and predictability.

P

Performance

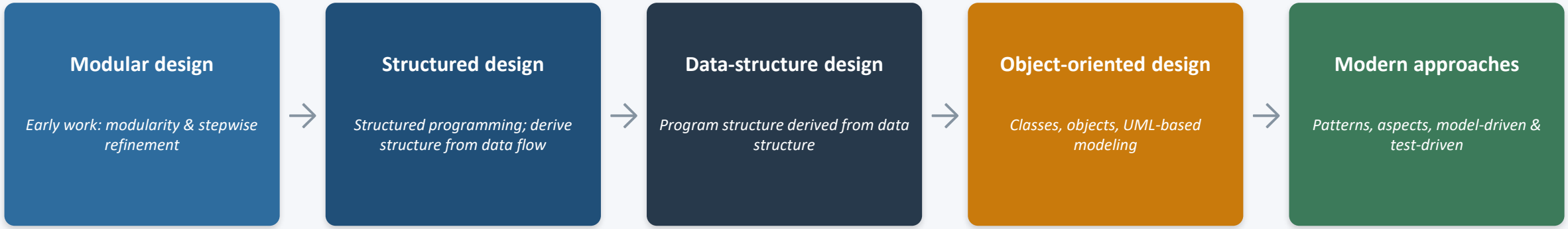
Measured by processing speed, response time, resource consumption, throughput, and efficiency.

S

Supportability

Extensibility, adaptability, serviceability — maintainability, testability, compatibility, configurability, and ease of installation.

How Design Thinking Has Evolved



What every design method shares

- A mechanism for translating the requirements model into a design representation.
- A notation for representing functional components and their interfaces.
- Heuristics for refinement and partitioning of the design.
- Quality criteria against which the design can be assessed.

Ten Fundamental Ideas

1. **Abstraction**
   Hide detail; focus on essentials.
2. **Architecture**
   Overall structure of the software.
3. **Patterns**
   Proven solutions to recurring problems.
4. **Separation of concerns**
   Divide a problem into distinct parts.
5. **Modularity**
   Break software into named components.
6. **Information hiding**
   Conceal internal detail inside modules.
7. **Functional independence**
   High cohesion, low coupling.
8. **Refinement**
   Elaborate detail step by step.
9. **Aspects**
   Handle crosscutting concerns.
10. **Refactoring**
    Improve structure without changing behavior.

Plus: object-oriented design concepts and design classes.

Managing Complexity by Hiding Detail

Procedural abstraction

- A named sequence of instructions with a specific, limited function.
- Example: the word “open” for a door implies a long sequence — walk to the door, reach out, grasp the knob, turn, pull.

Data abstraction

- A named collection of data that describes a data object.
- Example: “door” is a data abstraction with attributes such as door type, swing direction, opening mechanism, weight, and dimensions.

At the highest level of abstraction, a solution is stated in broad terms using the language of the problem environment. At lower levels, more procedural detail is added until the lowest level can be stated directly in source code.

The Overall Structure of the Software

Properties of an architectural design

- Structural properties — components and the way they are packaged and interact.
- Extra-functional properties — performance, capacity, security, reliability, and adaptability.
- Families of related systems — reuse of repeatable patterns and building blocks.

Models used to represent architecture

- Structural models — an organized collection of program components.
- Framework models — repeatable architectural design frameworks.
- Dynamic models — behavioral aspects; how the structure changes with events.
- Process models — the business or technical process the system supports.
- Functional models — the functional hierarchy of the system.

Reusing Proven Design Solutions

A design pattern describes a design structure that solves a particular design problem within a specific context and amid "forces" that may affect the way the pattern is applied and used.

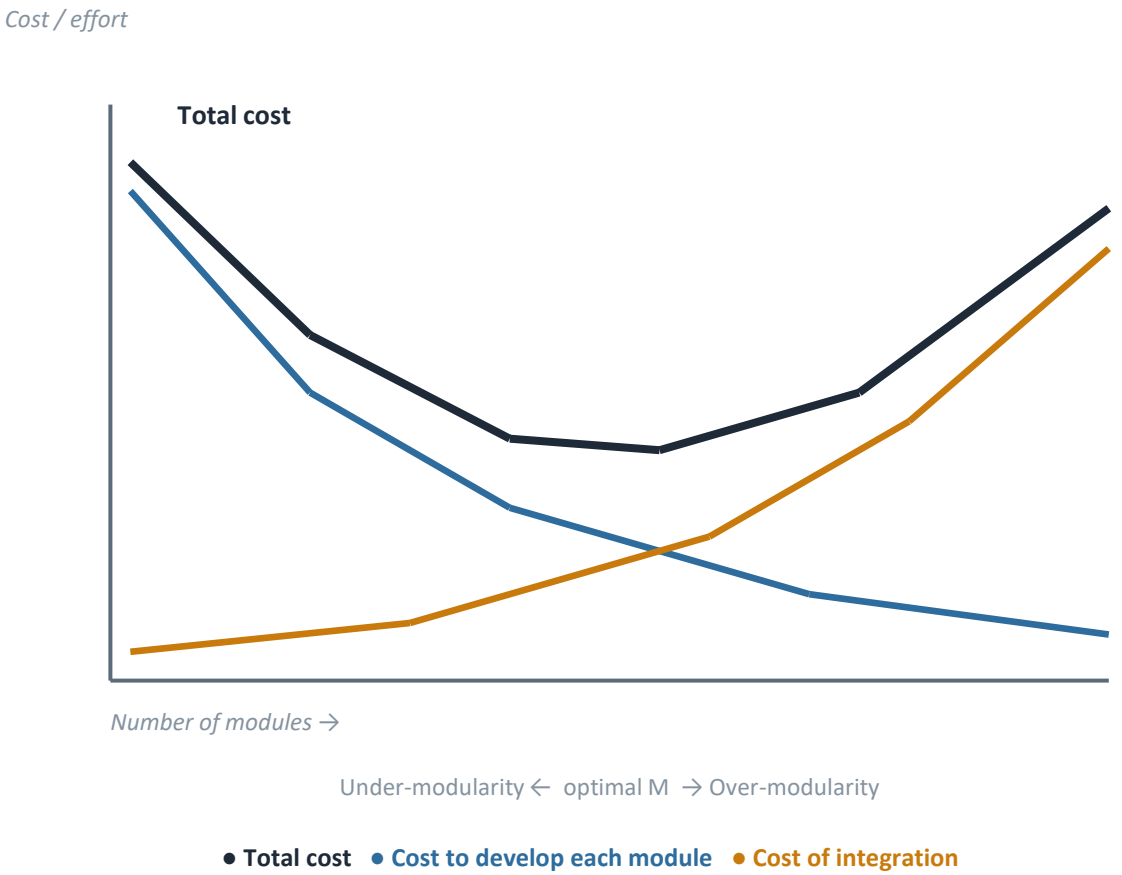
The intent of a pattern

- Determine whether the pattern is applicable to the current work.
- Determine whether it can be reused.
- Serve as a guide for developing a similar pattern that differs in function or structure.

Benefits

- Creates a shared vocabulary for communicating design insight.
- Captures experience so common problems are not re-solved from scratch.
- Supports reuse at the architectural, component, and interface levels.

Divide and Conquer



Separation of concerns

- A complex problem is easier to solve when divided into separate concerns.
- Each concern can be developed and validated independently.

Modularity

- Software is divided into separately named, addressable components (modules).
- Monolithic software is hard to grasp; too many modules raise integration cost — aim for the optimal M.

Modularity and Software Cost

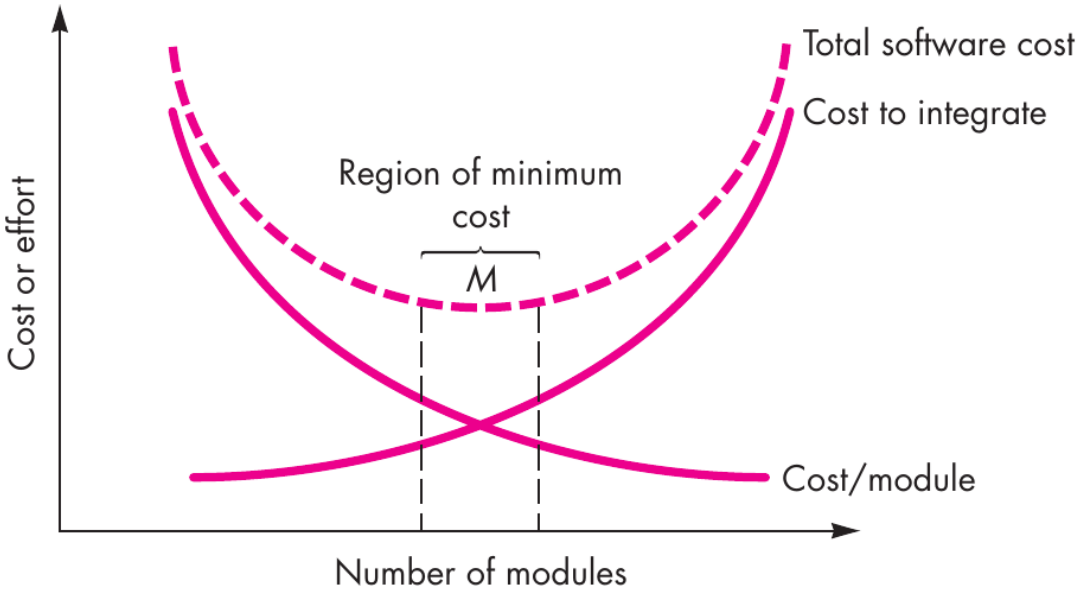
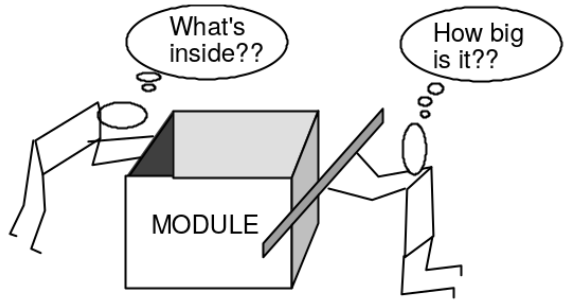
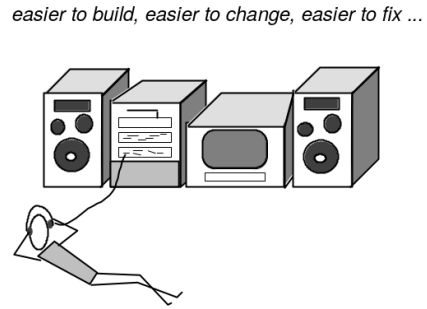


Figure 8.2 — Modularity and software cost (TB1, Ch. 8)

- More modules means smaller modules and a lower cost to develop each one, but the cost to integrate them rises.
- Total cost is lowest in the region around M — avoid under- and over-modularity.



A module hides what is inside it (Ch09)



Modular software is easier to build, change and fix (Ch09)

Keeping Modules Self-Contained

Information hiding

- Modules are specified so that internal details (procedure and data) are inaccessible to other modules.
- Only the information needed is passed through the interface.
- Limits the effect of changes and reduces error propagation.

Cohesion

- A qualitative indication of the degree to which a module focuses on just one thing.
- A cohesive module performs a single task with little interaction with other components.
- Goal: high cohesion.

Coupling

- An indication of the degree to which modules are interconnected.
- Tight coupling spreads errors and makes changes harder.
- Goal: low coupling — as low as is reasonable.

Elaborating and Improving the Design

Stepwise refinement

A top-down strategy: begin with a statement of function at a high level of abstraction, then elaborate it in successive steps until programming-language statements are reached. Abstraction and refinement are complementary — abstraction hides detail, refinement reveals it.

Aspects

Some concerns — such as security or logging — cut across many requirements and components. An aspect is a representation of a crosscutting concern, letting the design address it in one place instead of scattering it.

Refactoring

A reorganization technique that simplifies the design (or code) of a component without changing its function or behavior. It removes redundancy, unused elements, and poor structure to improve maintainability.

Refining Analysis Classes into Design Classes

Five types of design classes

- User interface classes — abstractions needed for human–computer interaction.
- Business domain classes — refined analysis classes for the business objects.
- Process classes — lower-level business abstractions to manage domain objects.
- Persistent classes — data stores (e.g. databases) that persist beyond execution.
- System classes — software management and control functions.

Characteristics of a well-formed design class

- Complete and sufficient — the full set of attributes and operations, and nothing more.
- Primitiveness — each operation focuses on one service.
- High cohesion — a small, focused set of responsibilities.
- Low coupling — collaborates with only the classes it must.

Design Classes and Messages in Pictures

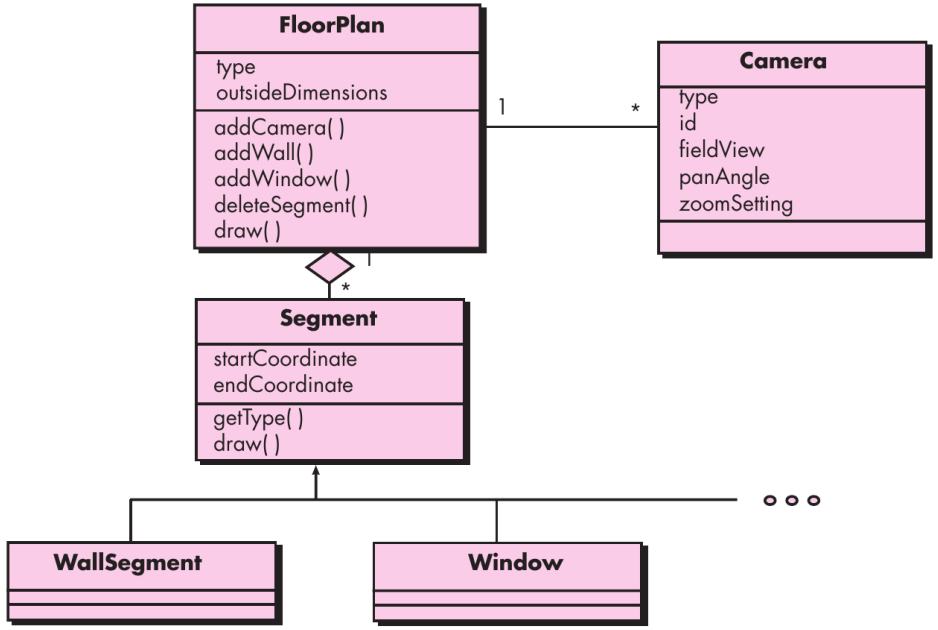
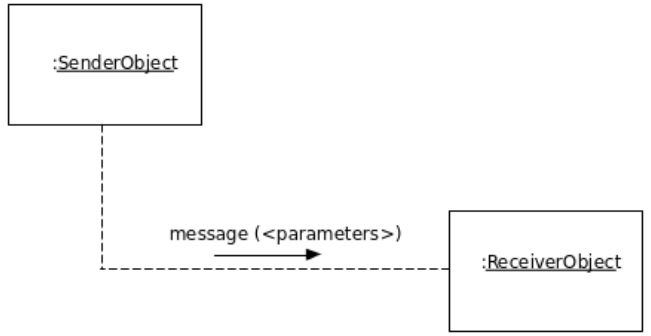


Figure 8.3 — Design class for FloorPlan and composite aggregation (TB1, Ch. 8)

- FloorPlan is refined into a design class: it is an aggregation of Segment objects and is associated with Camera.
- WallSegment and Window specialize Segment through inheritance.

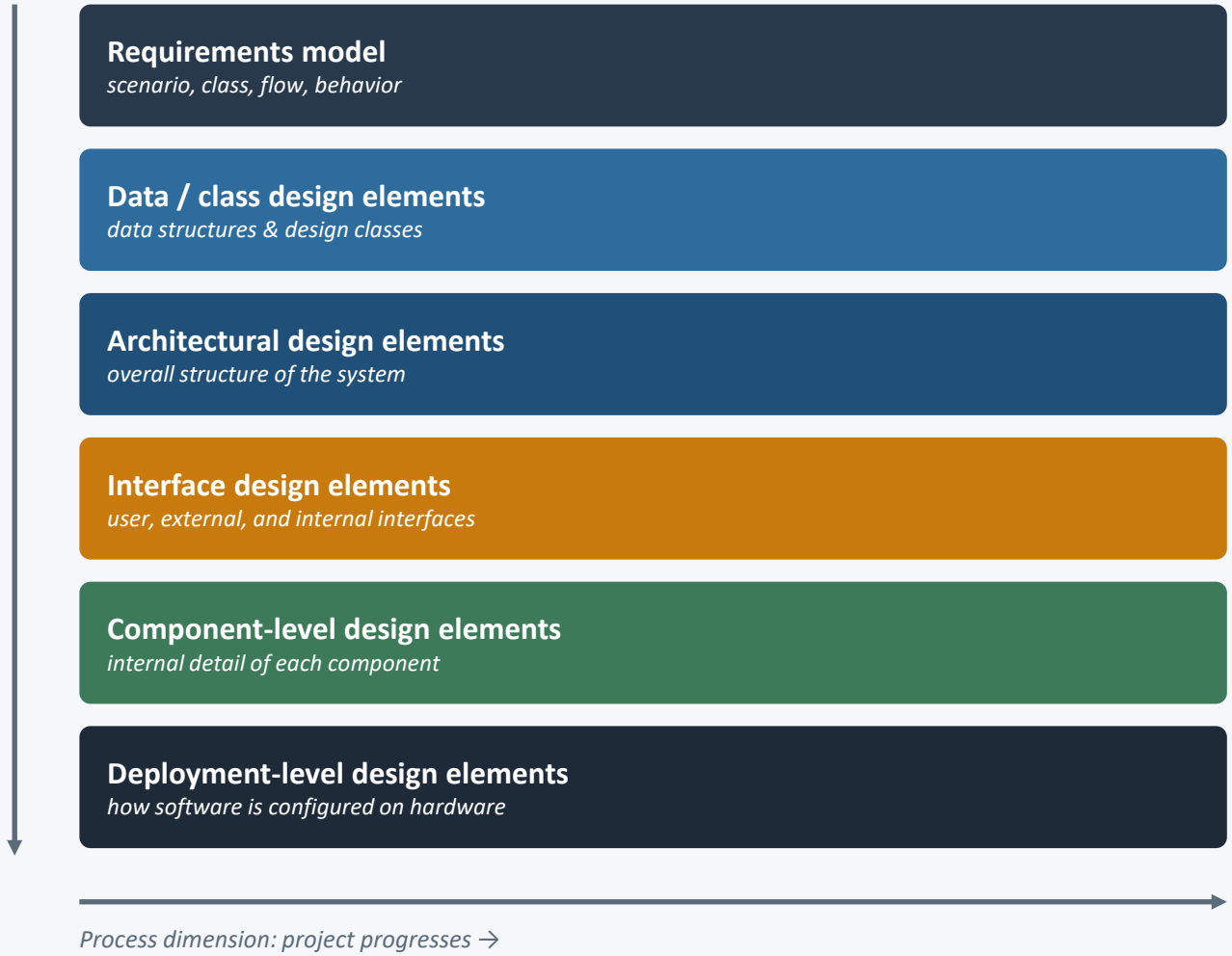


Objects collaborate by sending messages (Ch09)

Messages

- A message is how one object asks another to perform an operation.
- The receiver decides how to carry it out — the sender only needs the interface.

Two Dimensions: Process and Abstraction



Reading the model

- Process dimension: the design model evolves as design tasks are executed during the project.
- Abstraction dimension: the level of detail rises as each analysis element is refined into design.
- Elements are not strictly sequential — they are developed in parallel and revisited.
- All elements stay traceable to the requirements model.

Dimensions of the Design Model

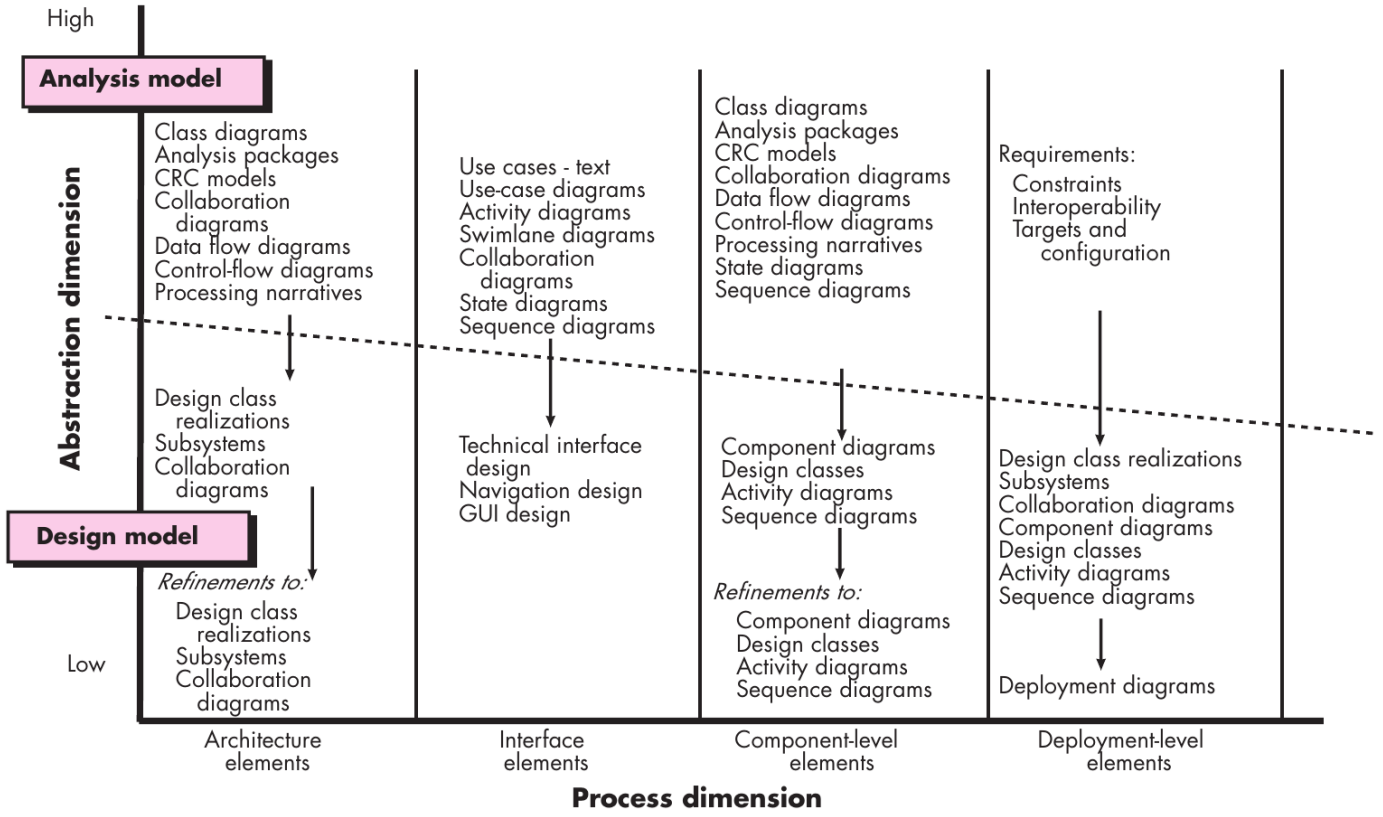


Figure 8.4 — Dimensions of the design model (TB1, Ch. 8)

Two dimensions

- Process dimension (horizontal): the design model evolves as design tasks are executed.
- Abstraction dimension (vertical): detail rises as analysis elements are refined into design elements.
- The dashed line shows that the analysis model blends gradually into the design model.
- Elements are developed in parallel and stay traceable to requirements.

Designing the Foundation

Data design elements

- Create a model of data and/or information at a high level of abstraction.
- The data model is refined into progressively more implementation-specific representations.
- At the architecture level: databases and files; at the component level: data structures.
- Well-structured data design simplifies program flow and makes processing more efficient.

Architectural design elements

- Give an overall view of the software, like a house's floor plan.
- Derived from the application domain (requirements model), analogous architectural styles and patterns, and the information in analysis models.
- Describe subsystems and the relationships among major components.

Designing the Details

Interface design elements

- User interface (UI) — screen layouts, navigation, and interaction.
- External interfaces to other systems, devices, and networks.
- Internal interfaces between design components.

Component-level design elements

- Fully describe the internal detail of each software component.
- Define data structures, algorithms, and the interface for every operation.
- Represented using UML diagrams or pseudocode.

Deployment-level design elements

- Show how software functionality and subsystems are allocated to the physical computing environment.
- Typically shown with UML deployment diagrams.

UML Interface and Component Diagrams

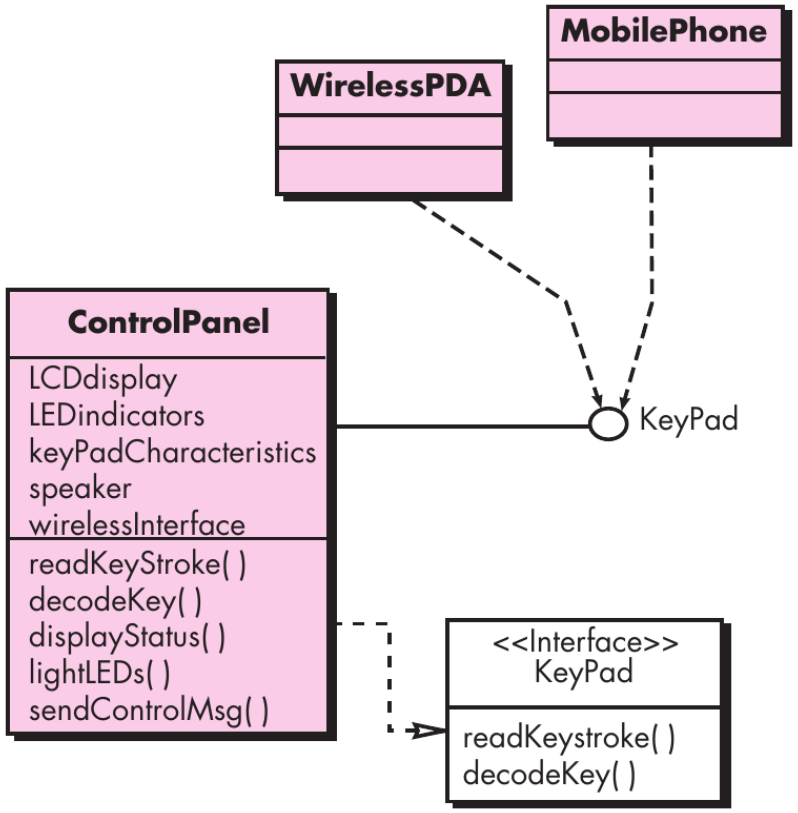


Figure 8.5 — Interface representation for ControlPanel (TB1, Ch. 8)

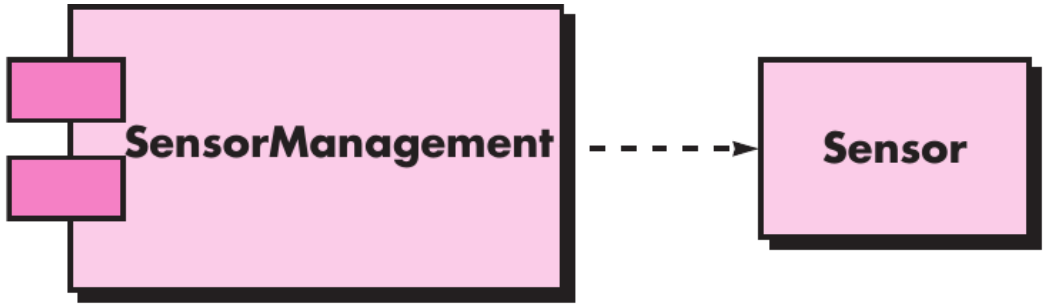


Figure 8.6 — A UML component diagram (TB1, Ch. 8)

- An interface is drawn as a labeled circle (or an <<interface>> class) listing the operations it provides.
- ControlPanel realizes the KeyPad interface; WirelessPDA and MobilePhone depend on it.
- A component such as SensorManagement is linked by a dashed arrow to the class it uses (Sensor).

UML Deployment Diagram for SafeHome

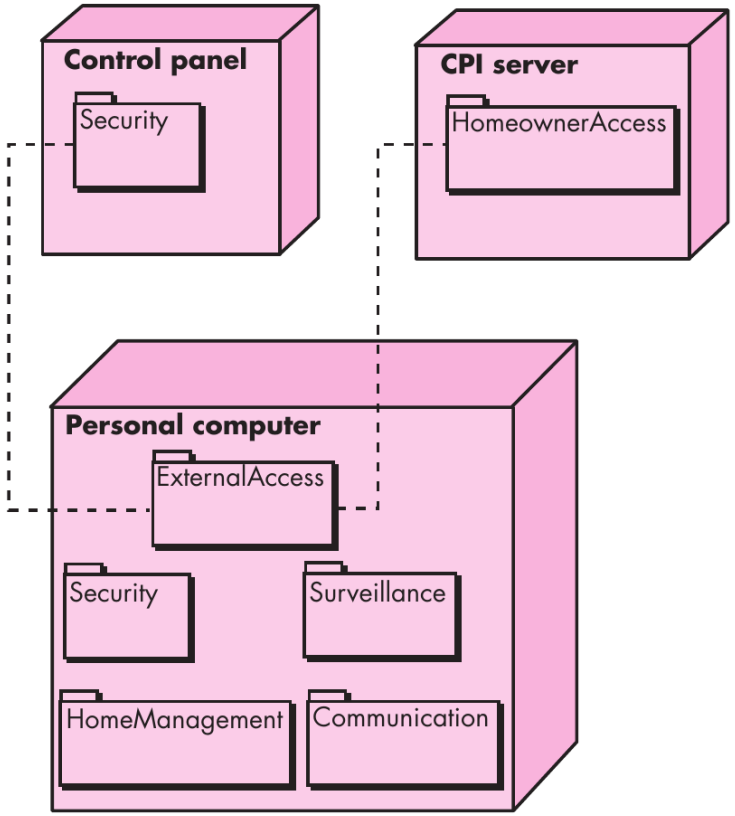


Figure 8.7 — A UML deployment diagram (TB1, Ch. 8)

What it shows

- Deployment diagrams show how software functionality and subsystems are allocated to the physical computing environment.
- Three computing environments appear: the personal computer, the control panel, and the CPI server.
- Each node lists the subsystems it hosts, such as Security, Surveillance, HomeManagement, and Communication.
- Descriptor form shows general environments; instance form later adds specific, named configurations.

KEY TAKEAWAYS

Design in a Nutshell

- **Design is where quality is built**

  It turns the requirements model into a blueprint for construction.

- **Good design meets guidelines and FURPS**

  Modular, understandable, and measured on functionality, usability, reliability, performance, supportability.

- **Core concepts guide every decision**

  Abstraction, architecture, patterns, modularity, information hiding, independence, refinement, refactoring.

- **The design model has five element types**

  Data, architectural, interface, component-level, and deployment-level — all traceable to requirements.